

Rekayasa Perangkat Lunak

Pertemuan 2 — Software Development Life Cycle (SDLC)

Teori & Praktik Industri · Adithya Marhaendra Kusuma, S.T., M.Kom.



Tujuan Pembelajaran

1

Konsep Dasar SDLC

Memahami konsep dasar SDLC dan fungsinya dalam mencegah kegagalan proyek IT yang berbiaya tinggi.

2

6 Tahapan SDLC

Menjelaskan keenam tahapan pengembangan perangkat lunak beserta contoh kasus nyata di industri logistik dan *Supply Chain*.

3

Analisis Model SDLC

Membedakan dan menganalisis model Waterfall, Agile, dan Spiral untuk memilih pendekatan terbaik sesuai kebutuhan proyek operasional.



Mengapa SDLC Sangat Penting?

Dalam skala industri besar seperti distribusi nasional atau FMCG, SDLC bukan sekadar panduan membuat aplikasi — ia adalah batas antara perusahaan yang untung dan perusahaan yang merugi karena sistemnya bermasalah.

Apa itu SDLC?

"Software Development Life Cycle (SDLC) adalah kerangka kerja (*framework*) terstruktur yang berisi langkah-langkah sistematis untuk merancang, membangun, menguji, dan memelihara perangkat lunak berkualitas tinggi."

SDLC bukan dokumen birokrasi — ia adalah **sistem imun** sebuah proyek teknologi. Tanpa SDLC, tim pengembang bekerja tanpa arah, anggaran membengkak, dan produk akhir sering kali tidak sesuai kebutuhan pengguna nyata di lapangan.

3 Alasan SDLC Krusial di Dunia Nyata

Mencegah Kerugian Operasional

Jika sistem sinkronisasi data gudang *error* karena tidak diuji, distribusi terhenti, barang kedaluwarsa, dan perusahaan menanggung kerugian besar yang bisa dihindari.

Efisiensi Waktu & Biaya

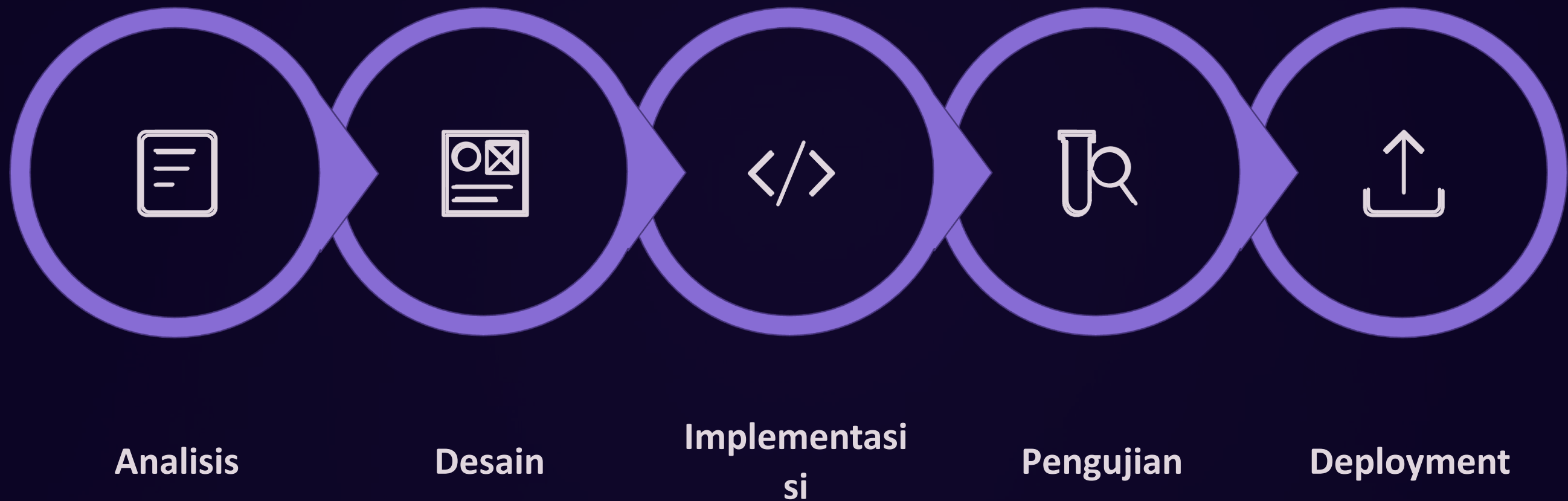
Memastikan perangkat lunak dibangun sesuai anggaran dan target peluncuran (*Time-to-Market*) yang sudah disepakati sejak awal bersama stakeholder.

Standardisasi Kerja

Memastikan *developer*, analis sistem, dan *user* (pengguna akhir) memiliki pemahaman dan acuan standar kerja yang sama di setiap fase proyek.

6 Tahapan SDLC

Setiap proyek perangkat lunak yang profesional melewati enam tahapan ini secara berurutan — dari analisis kebutuhan hingga pemeliharaan jangka panjang.



Keenam fase ini membentuk satu siklus penuh yang bisa berulang — karena perangkat lunak terus berkembang mengikuti kebutuhan bisnis yang dinamis.



TAHAP 1

Requirement Analysis

Analisis Kebutuhan — Fondasi Segalanya

Apa yang Terjadi di Fase Requirement Analysis?

Fase ini bukan sekadar mencatat keinginan *user* — melainkan melakukan studi kelayakan menyeluruh dari sisi operasional, teknis, dan finansial sebelum satu baris kode pun ditulis.

Aktivitas Utama

- Mengumpulkan spesifikasi kebutuhan dari semua stakeholder
- Mengidentifikasi masalah lapangan secara langsung
- Memastikan sistem yang akan dibangun layak secara teknis
- Menyusun *Business Requirement Document* (BRD) sebagai output resmi

Mengapa Ini Penting?

Kesalahan yang tidak terdeteksi di fase ini akan jauh lebih mahal untuk diperbaiki di fase-fase selanjutnya. Biaya perbaikan *bug* yang berasal dari kesalahan analisis bisa 10–100× lebih besar dibanding memperbaikinya di sini.

- ① Output: Business Requirement Document (BRD) yang menjadi kontrak teknis antara tim IT dan bisnis.



STUDI KASUS INDUSTRI — TAHAP 1

Kasus Nyata: Pelacakan OTA Truk Real- Time

Tim distribusi lapangan mengajukan permintaan fitur pelacakan On Time Arrival (OTA) truk secara *real-time*. Respons yang benar bukan langsung mengiyakan — melainkan melakukan evaluasi mendalam terlebih dahulu.

"Apakah infrastruktur server kita mampu menampung data GPS dari ribuan truk setiap detik secara serentak? Berapa biaya *bandwidth*-nya per bulan? Apakah ini sebanding dengan nilai bisnis yang dihasilkan?"



TAHAP 2

Design

Perancangan Sistem — Cetak Biru Sebelum Membangun

Dua Level Perancangan Sistem

Berbekal BRD dari tahap pertama, arsitek sistem mulai membuat cetak biru (blueprint) lengkap sebelum tim programmer menulis satu baris kode pun.

High- Level Design (HLD)

Merancang arsitektur makro sistem: pemilihan *server*, infrastruktur *cloud* (AWS/GCP/Azure), arsitektur *microservices*, dan alur data antar komponen besar.

Low-Level Design (LLD)

Merancang detail teknis: struktur *database*/ERD, alur logika aplikasi (*flowchart*), rancangan UI/UX per halaman, dan spesifikasi API antar modul.

Desain UI Adaptif: Supir vs. Manajer

Satu sistem logistik, dua antarmuka yang berbeda 180° — keduanya dirancang di fase ini.



Aplikasi Supir Truk

Tombol besar, antarmuka sederhana, dan navigasi satu tangan. Dirancang untuk dioperasikan saat berkendara di kondisi cahaya terang maupun malam hari.



Dashboard Control Tower

Padat informasi, penuh tabel dan grafik *fulfillment delivery* real-time. Dirancang untuk manajer yang memantau ratusan pengiriman sekaligus dari kantor pusat.

TAHAP 3

Implementation

Fase Konstruksi / Coding — Saat Desain Menjadi Kenyataan



Siapa yang Bekerja di Fase Implementation?

Fase konstruksi adalah saat tim *programmer* menerjemahkan seluruh dokumen desain menjadi **kode program nyata** yang bisa dijalankan. Setiap spesialis memiliki tanggung jawab yang jelas dan saling bergantung.



Front- End Developer

Membangun antarmuka pengguna (UI) yang dilihat dan diinteraksikan langsung oleh *user*. Menggunakan teknologi seperti React, Flutter, atau Vue.js.



Back-End Developer

Membangun logika bisnis, API, dan layanan server yang memproses data di balik layar. Menggunakan teknologi seperti Node.js, Django, atau Spring Boot.



Database Administrator

Membangun dan mengelola struktur basis data sesuai ERD, memastikan performa *query*, keamanan data, dan strategi *backup* yang andal.

✅ Output fase ini: Perangkat lunak atau fitur awal yang sudah bisa dioperasikan dan siap masuk ke fase pengujian.

TAHAP 4

Testing

Fase Pengujian Terstruktur — Temukan Bug Sebelum User Menemukannya



Mengapa Fase Testing Tidak Bisa Dilewati?

Testing adalah **garis pertahanan terakhir** sebelum perangkat lunak menyentuh pengguna nyata. Setiap *bug* yang lolos ke tahap produksi berpotensi menimbulkan kerugian finansial dan reputasi yang jauh lebih besar.

Functional Testing

Memverifikasi bahwa setiap fitur bekerja sesuai spesifikasi yang tertulis di BRD. Apakah tombol melakukan apa yang seharusnya? Apakah data tersimpan dengan benar?

Load Testing

Menguji ketahanan sistem saat dibanjiri ribuan pengguna secara bersamaan. Memastikan server tidak *crash* saat traffic puncak seperti hari raya atau promo nasional.

Security Testing

Mensimulasikan serangan siber untuk menemukan celah keamanan sebelum peretas menemukannya. Termasuk pengujian SQL Injection, XSS, dan autentikasi.

User Acceptance Testing (UAT)

Pengguna asli mencoba sistem secara langsung di lingkungan yang menyerupai kondisi nyata. Ini adalah "sidang akhir" sebelum sistem resmi dirilis ke publik.



STUDI KASUS INDUSTRI — TAHAP 4

Skenario Pengujian Ekstrem: Blank Spot

Sebelum aplikasi logistik dirilis, skenario kritis ini wajib diuji tuntas oleh tim QA.

"Apa yang terjadi jika supir menginput data serah terima barang di daerah pelosok yang tidak ada sinyal internet (*blank spot*)? Apakah aplikasi akan *crash*, atau data akan tersimpan di *cache* lokal dan terkirim otomatis saat sinyal kembali?"

- ⊗ Jika aplikasi *crash* dalam skenario ini, maka fase pengujian dianggap gagal dan tim harus kembali ke fase implementasi untuk perbaikan.



TAHAP 5

Deployment

Penerapan / Go-Live — Saat Sistem Menyentuh Dunia Nyata

Strategi Rilis yang Aman: Phased Rollout

Fase *deployment* bukan hanya tentang menekan tombol "publish" — ini tentang **strategi transisi** yang memastikan sistem berjalan stabil tanpa mengganggu operasional bisnis yang sedang berjalan.

1

Persiapan Go-Live

Konfigurasi server produksi, migrasi data, dan pembuatan SOP teknis untuk tim operasional dan pengguna akhir.

2

Rilis Regional (Jakarta)

Gunakan *Phased Rollout*: rilis terbatas di satu regional dulu. Pantau performa, kumpulkan feedback, dan pastikan sistem stabil.

3

Ekspansi Nasional

Jika regional pertama stabil, ekspansi bertahap ke regional lain. Hindari rilis serentak (*Big Bang*) untuk sistem kritis berskala nasional.

⚠️ Selalu hindari strategi *Big Bang Deployment* untuk sistem kritis. Satu masalah kecil dapat melumpuhkan seluruh operasional nasional secara sekaligus.



TAHAP 6

Maintenance

Pemeliharaan Sistem — Realitas Pekerjaan Teknis Sehari-hari

Maintenance: Fase Terlama dalam Siklus Hidup Software

Banyak yang mengira pekerjaan selesai setelah *deployment*. Faktanya, **maintenance** adalah fase yang paling panjang dan merupakan realitas pekerjaan teknis sehari-hari di perusahaan. Sebuah sistem enterprise bisa digunakan 5–15 tahun dan terus memerlukan pemeliharaan aktif.



Corrective Maintenance

Troubleshooting dan perbaikan *bug* yang muncul di lingkungan produksi nyata yang tidak terdeteksi saat pengujian.



Audit & Validasi Data

Memastikan integritas dan validitas data harian — kritis untuk sistem distribusi yang memproses ribuan transaksi setiap hari.



Adaptive Maintenance

Memperbarui sistem mengikuti perubahan regulasi pemerintah, upgrade infrastruktur teknologi, atau integrasi sistem baru dari mitra bisnis.

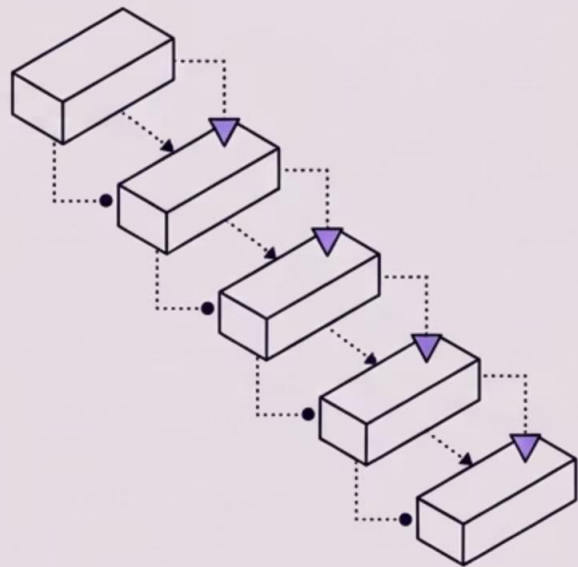
Rekap: 6 Tahapan SDLC dalam Satu Pandangan

Tahap	Nama	Output Utama	Pelaku Kunci
1	Requirement Analysis	Business Requirement Document (BRD)	Business Analyst, Stakeholder
2	Design	Arsitektur sistem, ERD, UI/UX Mockup	System Architect, UI/UX Designer
3	Implementation	Source code, build aplikasi	Front-end, Back-end, DBA
4	Testing	Laporan bug, UAT sign-off	QA Engineer, End User
5	Deployment	Aplikasi live di server produksi	DevOps, IT Ops
6	Maintenance	Patch, update, laporan audit	Tim IT, Support Engineer

Model-Model SDLC

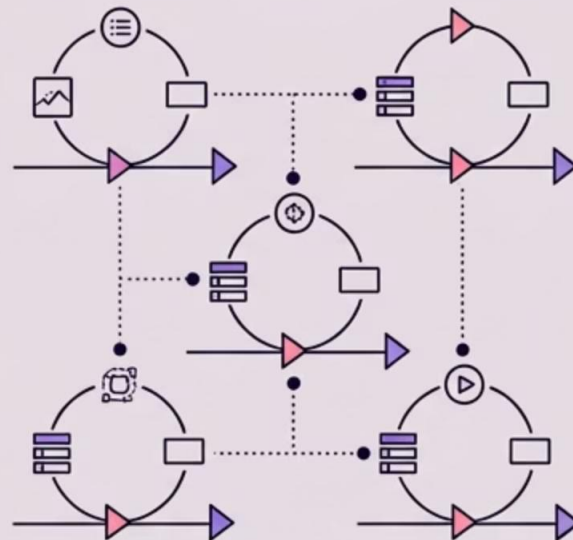
Tidak ada satu model SDLC yang cocok untuk semua jenis proyek. Pilihan model bergantung pada ukuran proyek, tingkat kepastian kebutuhan, toleransi risiko, dan kecepatan yang dibutuhkan untuk menghadirkan produk ke pasar.

MODEL WATERFALL



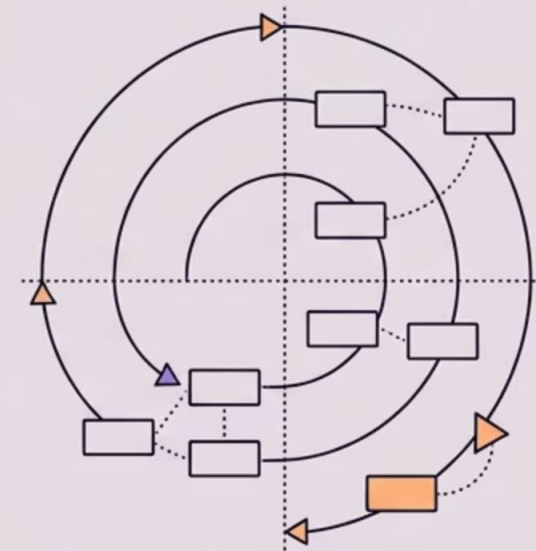
Linier, persyaratan tetap, pengiriman lama. Terbaik untuk sistem akuntansi/pajak.

MODEL AGILE



Iteratif, persyaratan fleksibel, pengiriman cepat. Terbaik untuk dasbor/e-commerce/startup.

MODEL SPIRAL



Hibrida Waterfall & Agile berisiko. Untuk sistem kritis taruhan tinggi seperti migrasi perbankan atau militer.



MODEL KLASIK

Model Waterfall

Pendekatan **linear dan berurutan** — seperti air terjun, setiap tahap harus selesai 100% sebelum tahap berikutnya dimulai. Model ini adalah standar industri perangkat lunak selama dekade 1970–1990an.

Karakteristik Model Waterfall

Kekuatan Waterfall

- Dokumentasi sangat lengkap dan terstruktur
- Mudah dikelola karena setiap fase memiliki deliverable yang jelas
- Cocok saat kebutuhan sudah pasti dan tidak berubah
- Lebih mudah untuk estimasi biaya dan waktu sejak awal

Kelemahan Waterfall

- Kaku — perubahan *requirement* di tengah jalan sangat mahal
- Pengguna hanya terlibat di awal dan akhir proyek
- Produk baru bisa digunakan setelah seluruh sistem selesai 100%
- Risiko tinggi: jika analisis awal salah, seluruh proyek bisa gagal

- ✓ Terbaik untuk: Sistem Keuangan/Akuntansi, Perpajakan, atau sistem pemerintahan dengan aturan yang baku dan tidak berubah-ubah.



MODEL MODERN

Model Agile

Pendekatan iteratif dan inkremental — proyek dipecah menjadi siklus kerja pendek (biasanya 2 minggu) yang disebut *Sprint*. Setiap *Sprint* menghasilkan fitur yang bisa langsung digunakan dan diuji oleh pengguna.

Filosofi Inti Agile: 4 Nilai Utama

Agile lahir pada 2001 dari *Agile Manifesto* yang ditulis oleh 17 praktisi perangkat lunak. Manifesto ini membalik cara berpikir tradisional tentang pengembangan software.

Individu & Interaksi

Lebih utama dari proses dan alat kerja

Working Software

Lebih utama dari dokumentasi yang komprehensif

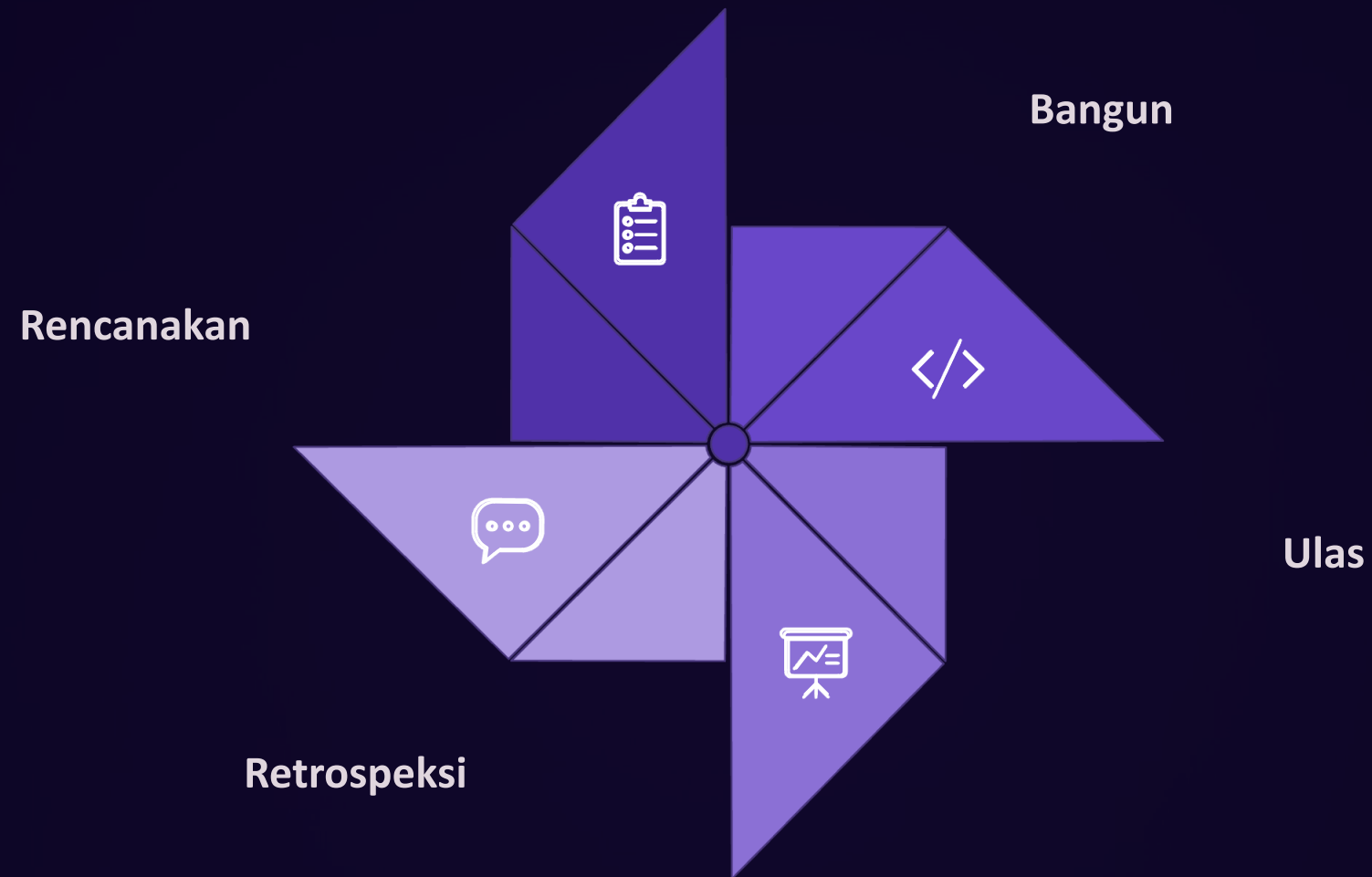
Kolaborasi Pelanggan

Lebih utama dari negosiasi kontrak yang kaku

Respons Perubahan

Lebih utama dari mengikuti rencana awal yang sudah usang

Cara Kerja Agile: Siklus Sprint



Siklus ini berulang terus menerus. Setiap iterasi menghasilkan perangkat lunak yang lebih baik dan lebih sesuai dengan kebutuhan nyata pengguna yang terus berkembang.

✅ Terbaik untuk: Dashboard operasional, e-commerce, aplikasi startup, atau proyek di mana kebutuhan pengguna belum sepenuhnya jelas sejak awal.

Waterfall vs. Agile: Perbandingan Langsung

Dimensi	 Waterfall	 Agile
Karakteristik	Linear. Tahapan berurutan ke bawah seperti air terjun	Iteratif. Dipecah dalam <i>Sprint</i> 2 mingguan
Fleksibilitas	Kaku. Semua kebutuhan harus <i>fixed</i> di awal	Fleksibel. Adaptif terhadap perubahan spesifikasi
Keterlibatan User	Hanya di fase awal (wawancara) dan akhir (UAT)	Sangat tinggi dan kontinu di setiap siklus
Waktu Rilis	Lama — menunggu seluruh sistem selesai 100%	Cepat — fitur dasar bisa langsung digunakan
Kecocokan	Sistem keuangan, perpajakan, regulasi pemerintah	Dashboard operasional, e-commerce, startup



MODEL HYBRID

Model Spiral

Ketika Kegagalan Bukan Pilihan

Model Spiral: Fleksibilitas + Kontrol + Analisis Risiko

Model Spiral menggabungkan **fleksibilitas Agile** dengan **kontrol dokumentasi ketat Waterfall**, dengan tambahan elemen kunci yang tidak dimiliki kedua model lainnya: **Analisis Risiko Mendalam** di setiap putaran iterasi.

4 Kuadran Spiral

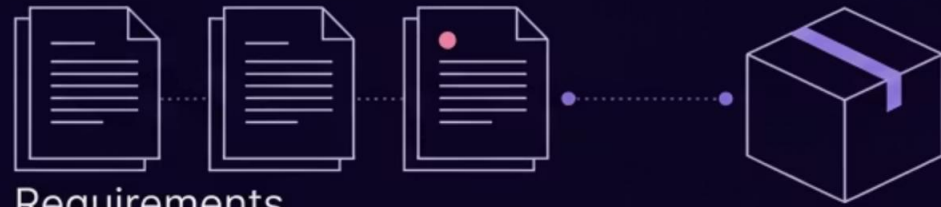
- **Perencanaan:** Tentukan tujuan dan alternatif
- **Analisis Risiko:** Identifikasi dan mitigasi risiko
- **Rekayasa:** Bangun dan uji prototype/produk
- **Evaluasi:** Review oleh customer sebelum iterasi berikutnya

Kapan Menggunakan Spiral?

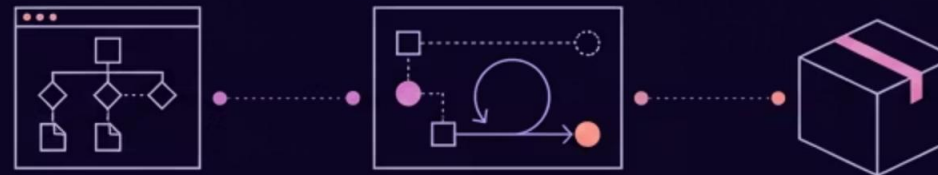
Cocok untuk proyek masif dengan risiko kegagalan fatal — di mana satu kesalahan berarti kerugian finansial triliunan rupiah atau bahkan mengancam nyawa manusia.

- Migrasi database perbankan inti
- Sistem alutsista militer
- Sistem navigasi penerbangan
- Infrastruktur energi nasional

Kapan Memilih Model SDLC yang Mana?



Waterfall: Kejelasan Persyaratan: Jelas & Tetap, Ukuran Proyek: Menengah, Toleransi Risiko: Rendah.



Agile: Kejelasan Persyaratan: Tidak Jelas & Berkembang, Ukuran Proyek: Kecil-Menengah, Toleransi Risiko: Menengah.



Spiral: Kejelasan Persyaratan: Sebagian Jelas tapi Risiko Tinggi, Ukuran Proyek: Sangat Besar, Toleransi Risiko: Nol.



Panduan Pemilihan Model SDLC Berdasarkan 3 Kriteria.

STUDI KASUS

Diskusi Kelas



Skenario: Startup Agrikultur Farm- to- Table

"Sebuah startup agrikultur memiliki tenggat waktu hanya **3 bulan** untuk meluncurkan aplikasi distribusi sayur segar (farm-to-table) sebelum kehabisan dana investor. Di sisi lain, mereka belum sepenuhnya tahu fitur spesifik apa yang paling disukai restoran sebagai pelanggan utama mereka."

- ② Berdasarkan skenario ini, metodologi SDLC apa yang paling tepat — Waterfall, Agile, atau Spiral? Berikan argumentasi teknis terkait efisiensi waktu, fleksibilitas perubahan *requirement*, dan manajemen risiko bisnis!

Analisis Skenario: Faktor- Faktor Kunci

Sebelum memilih model, identifikasi dulu kondisi nyata dari startup ini. Tiga variabel berikut adalah penentu utama pemilihan metodologi.

1

Waktu Sangat Terbatas

Hanya 3 bulan sebelum kehabisan dana. Model yang memakan waktu lama untuk menghasilkan produk pertama langsung tereliminasi.

2

Kebutuhan Belum Jelas

Requirement dari restoran belum diketahui secara pasti. Model yang mengharuskan semua spesifikasi final di awal akan gagal di sini.

3

Risiko Bisnis Sedang

Kegagalan berarti kehilangan kepercayaan investor dan tutup bisnis — serius, tetapi bukan risiko nyawa. Model Spiral terlalu kompleks untuk skala ini.



Jawaban yang Direkomendasikan: Agile

Dari ketiga variabel analisis, Agile adalah pilihan yang paling tepat untuk startup agrikultur ini. Berikut argumentasi teknisnya.

Mengapa Agile? Argumentasi Teknis

1 Efisiensi Waktu: MVP dalam 2–4 2–4 Minggu

Dengan Agile, tim bisa merilis *Minimum Viable Product* (MVP) — aplikasi dengan fitur dasar pemesanan sayur — dalam Sprint pertama (2 minggu). Investor bisa melihat produk nyata jauh sebelum batas 3 bulan tercapai.

2 Fleksibilitas Requirement: Belajar dari Restoran

Karena fitur ideal belum diketahui, Agile memungkinkan tim untuk **menguji hipotesis dengan restoran nyata** setiap Sprint, lalu mengubah arah (*pivot*) berdasarkan feedback aktual — bukan asumsi.

3 Manajemen Risiko Bisnis: Validasi Cepat

Risiko terbesar startup adalah membangun produk yang tidak dibutuhkan pasar. Agile memitigasi ini dengan siklus *build-measure-learn* yang cepat dan berulang, sehingga anggaran tidak habis untuk fitur yang salah.

Mengapa Bukan Waterfall atau Spiral?

✗ Waterfall Tidak Cocok

- Membutuhkan *requirement* lengkap di awal — padahal startup belum tahu fitur apa yang dibutuhkan restoran
- Produk baru bisa dilihat setelah bulan ke-3 atau lebih — terlalu lambat
- Jika fitur yang dibangun salah, tidak ada waktu untuk perbaikan

✗ Spiral Tidak Cocok

- Overhead analisis risiko formal terlalu berat untuk tim kecil startup dengan 3 bulan runway
- Dokumentasi dan proses Spiral dirancang untuk proyek berskala enterprise besar
- Biaya dan waktu setup Spiral melebihi kapasitas startup agrikultur ini

Key Takeaways Pertemuan 2

Tiga hal paling penting yang harus dibawa pulang dari pertemuan ini sebagai bekal praktis di dunia kerja.

SDLC adalah Fondasi

Setiap perangkat lunak profesional wajib melalui keenam tahapan SDLC. Melewati satu tahap saja dapat berakibat fatal bagi produk dan perusahaan.

Pilih Model Secara Kontekstual

Tidak ada model SDLC yang terbaik secara mutlak. Waterfall, Agile, dan Spiral masing-masing memiliki konteks penggunaan yang tepat berdasarkan kondisi proyek.

Industri adalah Laboratorium Laboratorium Terbaik

Kasus OTA truk, UI supir vs. manajer, dan pengujian *blank spot* membuktikan bahwa teori SDLC bukan abstrak — ia adalah panduan nyata yang menentukan keberhasilan sistem di lapangan.

Pertanyaan untuk Refleksi Mandiri

Gunakan pertanyaan-pertanyaan berikut untuk memperdalam pemahaman Anda sebelum pertemuan berikutnya.

Tentang Fase Testing

Bayangkan Anda QA Engineer di perusahaan e-commerce besar. Fitur apa saja yang akan Anda prioritaskan untuk diuji sebelum event Harbolnas 12.12 dengan jutaan transaksi serentak?

Tentang Pemilihan Model

Pemerintah ingin membangun sistem identitas digital nasional baru untuk 270 juta penduduk. Model SDLC apa yang paling tepat dan mengapa? Apa risiko terbesar yang harus diantisipasi?

Tentang Maintenance

Mengapa biaya maintenance sebuah sistem enterprise bisa melebihi biaya pembangunan awalnya? Strategi apa yang dapat diterapkan untuk menekan biaya ini secara jangka panjang?

Statistik Kegagalan Proyek IT Global

Data ini memperkuat mengapa memahami SDLC dengan benar bukan pilihan — melainkan keharusan bagi setiap profesional perangkat lunak.

31%

Proyek Dibatalkan

Dari seluruh proyek IT, hampir sepertiga tidak pernah selesai dan dibatalkan di tengah jalan — membuang seluruh investasi yang sudah dikeluarkan.

53%

Melebihi Anggaran

Lebih dari separuh proyek IT yang berhasil diselesaikan menghabiskan biaya 189% dari anggaran awal yang disetujui.

66%

Gagal Penuhi Target

Dua pertiga proyek IT gagal memenuhi tujuan bisnis awal yang dijanjikan kepada stakeholder.

\$2 T

Kerugian Global/ Tahun

Estimasi kerugian global akibat kegagalan proyek IT mencapai 2 triliun dolar AS setiap tahunnya.

Sumber: Standish Group CHAOS Report, IBM Systems Sciences Institute

Korelasi SDLC dengan Biaya Perbaikan Bug

Penelitian IBM menunjukkan bahwa biaya untuk memperbaiki kesalahan meningkat secara eksponensial seiring dengan semakin jauhnya fase tempat kesalahan itu ditemukan pertama kali.



Ditemukan di Requirement

Biaya perbaikan paling murah — hanya revisi dokumen BRD



Ditemukan di Design

10× lebih mahal dari fase requirement



Ditemukan di Testing

40× lebih mahal dari fase requirement



Ditemukan di Produksi

100× lebih mahal — plus kerugian operasional nyata

Implikasi Praktis

Inilah mengapa investasi waktu yang cukup di fase Requirement Analysis dan Design adalah keputusan bisnis yang paling rasional — bukan pemborosan waktu. Setiap jam yang dihabiskan untuk analisis yang baik di awal bisa menghemat ratusan jam perbaikan di kemudian hari.

 Shortcuts di fase awal SDLC hampir selalu menghasilkan *technical debt* yang jauh lebih mahal untuk dilunasi di kemudian hari.

KARIR

SDLC dalam Perjalanan Karir Anda

Pemahaman mendalam tentang SDLC bukan hanya nilai tambah di CV — ini adalah kompetensi inti yang dicari hampir semua perusahaan teknologi saat interview.



Peran Profesional dan Keterlibatan dalam SDLC



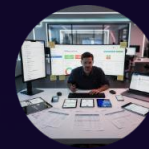
Business / System Analyst

Memimpin fase Requirement Analysis. Menjembatani kebutuhan bisnis dengan kemampuan teknis tim IT. Peran ini sangat dibutuhkan di perusahaan korporat besar.



Software Developer / Engineer

Bertanggung jawab di fase Implementation. Semakin baik memahami fase sebelumnya (Design), semakin sedikit *rework* yang dibutuhkan.



QA / Test Engineer

Menjaga kualitas produk di fase Testing. Profesi ini sering diremehkan namun krusial — QA yang baik menyelamatkan perusahaan dari skandal publik akibat *bug* produksi.



DevOps / Release Engineer

Mengeksekusi strategi Deployment dan menjaga stabilitas sistem di fase Maintenance. Menggabungkan keahlian pengembangan dan operasi infrastruktur.

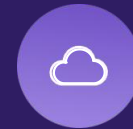
Tren SDLC di Industri Masa Kini

Dunia rekayasa perangkat lunak terus berkembang. Berikut adalah tren terkini yang mengubah cara industri mengimplementasikan SDLC.



DevSecOps

Integrasi keamanan (*Security*) sejak fase pertama SDLC — bukan hanya di Testing. Keamanan menjadi tanggung jawab seluruh tim, bukan hanya tim khusus security.



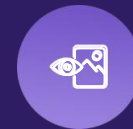
CI/ CD Pipeline

Continuous Integration / Continuous Deployment mengotomatiskan fase Testing dan Deployment, memungkinkan rilis fitur baru berkali-kali sehari tanpa gangguan layanan.



AI-Assisted Development

Alat seperti GitHub Copilot mulai mengubah fase Implementation — AI membantu *developer* menulis dan mereview kode lebih cepat, mengubah peran programmer secara fundamental.



Low- Code / No- Code

Platform seperti OutSystems dan Mendix memungkinkan pembuatan aplikasi bisnis tanpa menulis kode secara penuh — mempercepat fase Implementation untuk kasus-kasus tertentu.



Penutup: SDLC sebagai Kompas Profesional

Setiap sistem digital yang Anda gunakan hari ini — dari aplikasi ojek online, mobile banking, hingga platform belajar daring — lahir dari proses SDLC yang dijalankan dengan disiplin. Tugas Anda sebagai engineer masa depan adalah menjalankan proses ini dengan lebih baik, lebih cepat, dan lebih andal dari generasi sebelumnya.